

OpenFlow + Mininet lab

Learning objectives

- Understand how an OpenFlow controller programs flows into Open vSwitch (OVS) switches.
- Build a simple SDN topology in Mininet and connect it to an external Ryu controller.
- Implement a *learning switch* controller and extend it with simple ACL (firewall) functionality.
- Test, observe and explain flow entries in switches, and show how the controller reacts to traffic.

Quick environment setup (commands to run on the lab machine)

```
# install Open vSwitch (if not installed)
sudo apt-get install -y openvswitch-switch openvswitch-common
```

```
# install Ryu controller (python package)
```

```
Download Ryu source code from github
$ git clone https://github.com/osrg/ryu.git
```

```
NetworkX is a very important software to support your development of SDN controller.
$ pip install networkx
```

```
Install Ryu
$ cd ryu/
$ sudo python3 ./setup.py install
```

```
Execution and testing
$ ryu-manager
```

Topology used in this lab (what we will create)

We provide a small custom topology: two switches s1 and s2.

- s1 connects hosts h1 and h2.
- s2 connects hosts h3 and h4.
- s1 and s2 are linked.

IP addressing (static via Mininet defaults):

- h1 = 10.0.0.1
- h2 = 10.0.0.2
- h3 = 10.0.0.3
- h4 = 10.0.0.4

Ryu controller (OF 1.3) that behaves as a learning switch and implements simple firewall rules.

Files for the lab

1. custom_topo.py: Mininet custom topo (so they start the exact topology).
2. learning_firewall_ryu.py: Ryu app: learning switch + ACL examples (block ICMP and HTTP between specific hosts).

Paste these files on the lab machine.

#custom_topo.py

```
# custom_topo.py
from mininet.topo import Topo

class MyTopo( Topo ):
    def build( self ):
        s1 = self.addSwitch('s1')
        s2 = self.addSwitch('s2')

        # hosts with explicit IPs
        h1 = self.addHost('h1', ip='10.0.0.1/24')
        h2 = self.addHost('h2', ip='10.0.0.2/24')
        h3 = self.addHost('h3', ip='10.0.0.3/24')
        h4 = self.addHost('h4', ip='10.0.0.4/24')

        # links
        self.addLink(h1, s1)
        self.addLink(h2, s1)
        self.addLink(s1, s2)
        self.addLink(h3, s2)
        self.addLink(h4, s2)

topos = { 'mytopo': ( lambda: MyTopo() ) }
```

[illegible]

```
inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]
mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                        match=match, instructions=inst)
datapath.send_msg(mod)

def add_flow(self, datapath, priority, match, actions,
            idle_timeout=0, hard_timeout=0):
    parser = datapath.ofproto_parser
    ofproto = datapath.ofproto
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                        actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                            match=match, instructions=inst,
                            idle_timeout=idle_timeout,
                            hard_timeout=hard_timeout)
    datapath.send_msg(mod)

@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    msg = ev.msg
    datapath = msg.datapath
    dpid = datapath.id
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data)
    eth = pkt.get_protocols(ethernet.ethernet)[0]

    # ignore LLDP
    if eth.ethertype == ether_types.ETH_TYPE_LLDP:
        return

    src = eth.src
    dst = eth.dst

    self.mac_to_port.setdefault(dpid, {})
    self.mac_to_port[dpid][src] = in_port

    # examine IP/TCP/ICMP headers for firewall rules
    ip_pkt = pkt.get_protocol(ipv4.ipv4)
    tcp_pkt = pkt.get_protocol(tcp.tcp)
    icmp_pkt = pkt.get_protocol(icmp.icmp)

    # Example firewall rules:
    # 1) block ICMP from 10.0.0.1 -> 10.0.0.3
    # 2) block TCP dst port 80 (HTTP) from 10.0.0.1 -> 10.0.0.3
    if ip_pkt:
        src_ip = ip_pkt.src
        dst_ip = ip_pkt.dst
```

```

        if icmp_pkt:
            if src_ip == '10.0.0.1' and dst_ip == '10.0.0.3':
                self.logger.info("Dropping ICMP %s -> %s", src_ip, dst_ip)
                match = parser.OFPMatch(eth_type=0x0800, ipv4_src=src_ip,
                                         ipv4_dst=dst_ip, ip_proto=1)
                # install a short-lived drop flow (no actions)
                self.add_flow(datapath, priority=20, match=match, actions=[],
idle_timeout=30)
                return

            if tcp_pkt:
                if src_ip == '10.0.0.1' and dst_ip == '10.0.0.3' and tcp_pkt.dst_port == 80:
                    self.logger.info("Dropping HTTP %s -> %s", src_ip, dst_ip)
                    match = parser.OFPMatch(eth_type=0x0800, ipv4_src=src_ip,
                                             ipv4_dst=dst_ip, ip_proto=6, tcp_dst=80)
                    self.add_flow(datapath, priority=20, match=match, actions=[],
idle_timeout=30)
                    return

        # Learning switch behavior (install flows for known dst)
        if dst in self.mac_to_port[dpid]:
            out_port = self.mac_to_port[dpid][dst]
        else:
            out_port = ofproto.OFPP_FLOOD

        actions = [parser.OFPACTIONOutput(out_port)]

        if out_port != ofproto.OFPP_FLOOD:
            match = parser.OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
            self.add_flow(datapath, priority=10, match=match, actions=actions,
idle_timeout=30)

        data = None
        if msg.buffer_id == ofproto.OFP_NO_BUFFER:
            data = msg.data

        out = parser.OFPPacketOut(datapath=datapath,
                                   buffer_id=msg.buffer_id,
                                   in_port=in_port, actions=actions, data=data)
        datapath.send_msg(out)

```

Terminal A: Start the Ryu controller

Open a terminal and run:

```

# (optional) Make a folder for lab files, then paste learning_firewall_ryu.py inside
# run ryu (explicitly set listen port 6633 to match Mininet remote controller below)
ryu-manager --ofp-tcp-listen-port 6633 learning_firewall_ryu.py

```

Keep this terminal open to see controller logs (it prints info about blocked packets and flow installs).

If ryu-manager is not in PATH, run `python3 -m ryu` or `sudo ryu-manager` depending on the install.

Terminal B: Start Mininet with the custom topology and connect to the Ryu controller

Put `custom_topo.py` where you run the following command and run Mininet:

```
sudo mn --custom custom_topo.py --topo=mytopo \
--controller=remote,ip=127.0.0.1,port=6633 \
--switch=ovsk,protocols=OpenFlow13 --mac --arp
```

This opens the mininet> CLI.

In the Mininet CLI: basic checks & tests

1. Check connectivity (should be allowed unless the firewall blocks):

```
mininet> pingall
```

2. Inspect flows on switches (run in another terminal or prefix with `sudo`):

outside Mininet, list flows in s1

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s2
```

3. Demonstrate the firewall behaviour:

- Start a simple HTTP server on h3:

```
mininet> h3 python3 -m http.server 80 &
```

- Try to curl from h1 to h3:

```
mininet> h1 curl -sS 10.0.0.3 || echo "curl failed"
```

Because `learning_firewall_ryu.py` blocks HTTP from h1→h3, this should fail (or hang), while other host pairs should be able to fetch pages.

- Try ping:

```
mininet> h1 ping -c3 10.0.0.3
```

ICMP (ping) from h1→h3 is also blocked by the example firewall, so this should fail. Ping from h2→h3 should succeed:

```
mininet> h2 ping -c3 10.0.0.3
```

4. Observe controller logs in Terminal A; when an ACL rule triggers, you should see Dropping ICMP ... or Dropping HTTP

5. Clean flows and repeat if you want:

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
```

```
sudo ovs-ofctl -O OpenFlow13 del-flows s2
```

Task

Task: Starting from the provided `learning_firewall_ryu.py`:

1. **Understand:** Run the lab, show that learning switch behavior works, and that the example ACLs block h1→h3 for ICMP and HTTP.
2. **Modify** the controller to add the following features:
 - A configurable ACL list (in the code) where each rule is: (src_prefix, dst_prefix, ip_proto, dst_port, action), supporting allow and deny. You should be able to add new rules easily (e.g., a Python list at the top of the file).
 - Logging of every matched ACL rule to the Ryu log with the fields matched (src/dst/proto/port/decision).
 - Apply ACL rules before learning-switch installs to ensure forbidden traffic is dropped early.

3. **Test:** Provide a short test plan and show CLI evidence (ping/curl/wget or iperf) that demonstrates each ACL rule working.

Hints & common troubleshooting

- If Mininet switches do not connect to Ryu: check ports and that Ryu is listening on 6633 (or change Mininet to point at the port Ryu uses). Use `netstat -ltnp | grep ryu` to check.
- If flows don't show with `ovs-ofctl`, ensure you use OpenFlow13 option: `-O OpenFlow13 dump-flows s1`.
- If the controller logs nothing, ensure you started `ryu-manager` before `mn` and that you used `--controller=remote,ip=127.0.0.1,port=6633`.
- To see packet-ins and messages in Ryu, run `ryu-manager` with `--verbose`.
- If `python3 -m http.server 80` fails on the host inside Mininet (permission), use a high port like 8000 and request that port from client (`curl 10.0.0.3:8000`).